

Clase 005 — Linux esencial para seguridad: filesystem, permisos y usuarios

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Michael Kerrisk, The Linux Programming Interface*
🕒 Duración estimada: **110 min** · Nivel: **Fundamentos**

Objetivo

Dominar el modelo de archivos, permisos y usuarios de Linux, que es la base de la mayoría de escaladas de privilegios y de las medidas de *hardening*. Al terminar sabrás leer y modificar permisos, entender usuarios y grupos, y reconocer configuraciones peligrosas como binarios SUID.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Navegar** la jerarquía FHS y ubicar archivos clave de seguridad.
- 2. **Interpretar** y modificar permisos en notación simbólica y octal.
- 3. **Gestionar** usuarios, grupos y sus archivos de definición.
- 4. **Identificar** permisos especiales (SUID, SGID, sticky) y su riesgo.
- 5. **Aplicar** el principio de mínimo privilegio en el filesystem.

Temas

#	Tema	Por qué importa
1	FHS	Saber dónde vive cada cosa acelera todo
2	<code>/etc/passwd</code> y <code>/etc/shadow</code>	Fuente de usuarios y hashes de contraseña
3	Permisos <code>rwX</code>	Núcleo del control de acceso Unix
4	Notación octal	Forma compacta y examinable
5	Usuarios y grupos	UID/GID, grupos primarios y secundarios
6	SUID/SGID/sticky	Vector clásico de escalada de privilegios
7	<code>chmod</code> , <code>chown</code> , <code>umask</code>	Modificar y establecer permisos por defecto
8	ACLs	Control fino más allá de <code>rwX</code>

Definiciones y características

- **FHS (Filesystem Hierarchy Standard)**: convención de directorios (`/etc` , `/bin` , `/var` , `/home` ...).
Clave: `/etc` guarda configuración, `/var/log` los logs.

- **Permisos rwx**: lectura, escritura y ejecución para propietario, grupo y otros. Clave: en un directorio, `x` significa "poder entrar".
- **/etc/shadow**: almacena los hashes de las contraseñas, legible solo por root. Clave: objetivo de cracking offline.
- **SUID**: bit que hace que un ejecutable corra con los privilegios del propietario (a menudo root). Clave: un SUID mal puesto = escalada a root.
- **umask**: máscara que resta permisos a los archivos recién creados. Clave: define la exposición por defecto.
- **ACL (Access Control List)**: permisos por usuario/grupo adicionales a rwx. Clave: `getfacl` / `setfacl` para control granular.

Herramientas y preparación

Trabaja en tu VM Kali o cualquier Linux del laboratorio. Necesitas una terminal y una cuenta con `sudo`. Comandos base: `ls -l`, `chmod`, `chown`, `id`, `stat`, `find`, `getfacl` / `setfacl`. Ten a mano las páginas de manual (`man chmod`).

Laboratorio guiado

1. Explorar el FHS:

```
bash ls -la / ; ls -l /etc/passwd /etc/shadow
```

Observa que `shadow` no es legible por usuarios normales. 2. **Leer permisos**. Crea un archivo y examínalo:

```
bash touch prueba.txt ; stat prueba.txt ; ls -l prueba.txt
```

Descompón la cadena `-rw-r--r--` en propietario/grupo/otros. 3. **Cambiar permisos** en simbólico y octal:

```
bash chmod u+x prueba.txt ; ls -l prueba.txt chmod 640 prueba.txt ; ls -l prueba.txt
```

4. Usuarios y grupos. Crea un usuario de prueba (con sudo) y revisa su identidad:

```
bash sudo useradd -m alumno ; id alumno ; grep alumno /etc/passwd
```

5. Comprobar tu umask:

```
bash umask ; touch nuevo.txt ; ls -l nuevo.txt
```

6. Buscar binarios SUID en el sistema (reconocimiento de escalada):

```
bash find / -perm -4000 -type f 2>/dev/null
```

Anota los que aparezcan; muchos son legítimos (`sudo`, `passwd`). 7. **ACLs**. Da acceso puntual a `alumno` sobre un archivo:

```
bash setfacl -m u:alumno:r prueba.txt ; getfacl prueba.txt
```

Ejercicios

1. Traduce a octal: `rw-r-x---`, `rw-rw-r--`, `r-----`.
2. Explica qué significa el permiso `x` en un directorio frente a en un archivo.

3. Crea un directorio compartido por un grupo donde los archivos nuevos hereden el grupo (pista: SGID).
4. Configura un directorio "temporal" donde cada usuario solo pueda borrar sus propios archivos (sticky bit).
5. Investiga por qué `/etc/shadow` tiene permisos `640 root:shadow` y qué pasaría con `644`.
6. Con `find`, localiza archivos con escritura para "otros" en `/etc` y explica el riesgo.

Reto verificable

Configura una carpeta de proyecto compartida por un grupo `equipo` con estas propiedades: los miembros pueden crear y editar archivos, los archivos nuevos pertenecen automáticamente al grupo, y nadie puede borrar archivos ajenos. Documenta los permisos aplicados.

Criterio de aceptación: `ls -ld` de la carpeta muestra SGID y sticky bit activos; un segundo usuario del grupo puede crear un archivo que queda con grupo `equipo`, pero no puede borrar el del otro. Verificable con dos cuentas de prueba.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>Permission denied</code> al entrar a un directorio	Falta el bit <code>x</code> en el directorio. Añádalo con <code>chmod +x</code> .
<code>chmod 777</code> "para que funcione"	Antipatrón peligroso: da control total a todos. Usa el mínimo necesario.
No poder leer <code>/etc/shadow</code> como usuario	Es correcto: solo root/grupo shadow. Usa <code>sudo</code> si tienes permiso.
SUID root en un script propio no eleva	El kernel ignora SUID en scripts por seguridad; solo aplica a binarios.
Permisos correctos pero acceso denegado	Puede haber una ACL o SELinux/AppArmor. Revisa <code>getfacl</code> y el LSM.

Preguntas frecuentes

 **¿Octal o simbólico?** Ambos. Octal es compacto para asignar permisos completos; simbólico es cómodo para añadir/quitar un bit sin tocar el resto.

 **¿Por qué los binarios SUID son peligrosos?** Porque ejecutan con privilegios del propietario. Si un SUID root tiene un fallo o permite ejecutar comandos, un usuario normal puede volverse root. GTFOBins cataloga muchos casos.

 **¿root puede saltarse los permisos?** Sí, root ignora los permisos `rxw` tradicionales. Por eso limitar el uso de root y las escaladas es central en seguridad.

 **¿Las ACLs sustituyen a `rxw`?** No, las complementan. `rxw` sigue siendo la base; las ACLs añaden entradas por usuario/grupo cuando `rxw` no basta.

Referencias

- Michael Kerrisk, *The Linux Programming Interface* (caps. de archivos y permisos).
- `man 1 chmod`, `man 5 passwd`, `man 5 shadow`, `man 1 setfacl`
- Filesystem Hierarchy Standard — <https://refspecs.linuxfoundation.org/fhs.shtml>
- GTFOBins — <https://gtfobins.github.io/>

Siguiente clase

Clase 006 - Línea de comandos Linux avanzada: grep, sed, awk, pipes y procesos